D&D 3.5e Unity Project — Code Architecture Analysis

Date: May 14, 2026
Project: `/home/ubuntu/dnd35prototype`
Total Files: 355 `.cs` files | **164,603 total lines of C#**

---

1) File Size Analysis — Files Over 1,000 Lines

#	File	Lines	Methods	Severity
1	`Core/GameManager.cs`	**23,248**	~608	🔴 Critical
2	`Character/CharacterController.cs`	**11,229**	~498	🔴 Critical
3	`Character/CharacterStats.cs`	4,749	—	🟡 High
4	`UI/PreCombatInventoryUI.cs`	4,634	—	🟡 High
5	`UI/CombatUI.cs`	4,603	—	🟡 High
6	`UI/CharacterCreationUI.cs`	3,298	—	🟡 Medium
7	`Magic/SpellcastingComponent.cs`	2,748	—	🟡 Medium
8	`CombatSystems/GrappleSystem.cs`	2,024	—	🟡 Medium
9	`Store/StoreUI.cs`	2,017	—	🟡 Medium
10	`Services/AIService.cs`	1,989	—	🟡 Medium
11	`CombatSystems/SupportActions.cs`	1,989	—	🟡 Medium
12	`UI/SpellPreparationUI.cs`	1,826	—	🟡 Medium
13	`Inventory/ItemDatabase.cs`	1,765	—	🟡 Medium
14	`Character/NPCDatabaseCustom.cs`	1,663	—	🟡 Medium
15		1,597	—	🟢 Low

#	File	Lines	Methods	Severity
	`Core/GameManager_NewSpells.cs`			
16	`UI/CharacterSheetUI.cs`	1,566	—	🟢 Low
17	`Core/SceneBootstrap.cs`	1,470	—	🟢 Low
18	`Character/FeatDefinitions.cs`	1,407	—	🟢 Low
19	`UI/SpellSelectionUI.cs`	1,283	—	🟢 Low
20	`UI/RandomEncounterGeneratorUI.cs`	1,153	—	🟢 Low
21	`CombatSystems/StandardManeuvers.cs`	1,119	—	🟢 Low
22	`UI/Panels/ActionButtonPanel.cs`	1,096	—	🟢 Low
23	`UI/LootCollectionUI.cs`	1,080	—	🟢 Low
24	`Services/CombatFlowService.cs`	1,079	—	🟢 Low
25	`Inventory/ItemData.cs`	1,044	—	🟢 Low
26	`Tests/AI/AIProfileFrameworkTests.cs`	1,022	—	🟢 Low
27	`Magic/SpellCaster.cs`	1,015	—	🟢 Low

GameManager totals (all partial classes combined): 28,437 lines

---

2) Code Duplication — Specific Findings

🔴 HIGH PRIORITY — Attack Feat Bonus Calculation (3x Duplication)

Location: `CharacterController.cs` — lines ~4313-4391 (Single Attack), ~4566-4620 (Full Attack), ~5108-5243 (Dual Wield)

The same feat-bonus block is duplicated **3 times** almost identically:

```
Duplicated in each attack path:
- Power Attack check           (3 copies)
- Point Blank Shot check       (3 copies)
- Weapon Focus/Specialization  (3 copies)
- Weapon Finesse check         (3 copies)
- Combat Expertise check       (3 copies)
- Improved Critical check      (3 copies)
```

Total: ~33 feat-related checks duplicated across 3 attack methods

Recommended Refactoring:

```
// NEW: Extract to CharacterController or a new AttackCalculator service
public struct AttackModifiers
{
    public int AttackBonus;
    public int DamageBonus;
    public int CritThreatMin;
    public int CritMultiplier;
    public bool UsesDex;
    public string LogSummary;
}

public AttackModifiers CalculateFeatModifiers(
    ItemData weapon, bool isMelee, bool isRanged,
    RangeInfo rangeInfo, bool isOffHand = false)
{
    var mods = new AttackModifiers();

    // Power Attack
    if (isMelee && Stats.HasFeat("Power Attack") && PowerAttackValue > 0
        && !WeaponDisablesStrengthDamageBonuses(weapon))
    {
        mods.AttackBonus -= PowerAttackValue;
        mods.DamageBonus += PowerAttackValue * (weapon.IsTwoHanded ? 2 : 1);
    }

    // Point Blank Shot
    if (isRanged && Stats.HasFeat("Point Blank Shot")
        && rangeInfo?.DistanceFeet <= 30)
    {
        mods.AttackBonus += 1;
        mods.DamageBonus += 1;
    }

    // Weapon Focus / Specialization
    mods.AttackBonus += FeatManager.GetWeaponFocusBonus(Stats, weapon?.Name ?? "Un-
armed");
    mods.DamageBonus += FeatManager.GetWeaponSpecializationBonus(Stats, weapon?.Name ?
? "Unarmed");

    // Weapon Finesse
    if (isMelee && FeatManager.ShouldUseWeaponFinesse(Stats, weapon))
        mods.UsesDex = true;

    // Combat Expertise
    if (isMelee && Stats.HasFeat("Combat Expertise") && Stats.CombatExpertiseValue >
0)
        mods.AttackBonus -= Stats.CombatExpertiseValue;

    // Improved Critical
    mods.CritThreatMin = FeatManager.GetAdjustedCritThreatMin(Stats, weapon.CritThreat
Min);

    return mods;
}
```

Then `PerformSingleAttack()`, `PerformFullAttack()`, and `PerformDualWieldAttack()` all call:

```
var featMods = CalculateFeatModifiers(weapon, isMelee, isRanged, rangeInfo);
```

🔴 HIGH PRIORITY — PC vs NPC Spell Casting Duplication

Location: `GameManager.cs`
- `PerformSpellCast()` — **565 lines** (line 13384–13949)
- `TryNPCPerformSpellCast()` — **167 lines** (line 22077–22244)

Both contain **duplicate logic** for:
- Blink spell failure checks (20% caster failure, 50% target failure)
- Concentration checks
- Spell resistance rolls
- Area-of-effect dispatching

The NPC version is a condensed copy of the PC version. Both have identical Blink miss-chance blocks.

Recommended Refactoring:

```
// NEW: SpellCastingService.cs or GameManager_SpellResolution.cs

public class SpellResolutionResult
{
    public bool Fizzled;
    public bool TargetEvaded;
    public string FizzleReason;
}

public SpellResolutionResult ResolvePreCastChecks(
    CharacterController caster, CharacterController target,
    SpellData spell, SpellcastingComponent spellComp)
{
    var result = new SpellResolutionResult();

    // Blink caster failure (20%)
    if (caster.HasActiveBlinkEffect)
    {
        int roll = UnityEngine.Random.Range(1, 101);
        if (roll <= 20)
        {
            result.Fizzled = true;
            result.FizzleReason = $"Blink spell failure (rolled {roll} ≤ 20%)";
            return result;
        }
    }

    // Blink target evasion (50%)
    if (target != null && target != caster
        && !spell.IsAreaSpell && target.HasActiveBlinkEffect)
    {
        int roll = UnityEngine.Random.Range(1, 101);
        if (roll <= 50)
        {
            result.TargetEvaded = true;
            result.FizzleReason = $"Target is ethereal (rolled {roll} ≤ 50%)";
            return result;
        }
    }

    return result;
}
```

🟡 MEDIUM PRIORITY — Combat Log Verbosity (605 calls in GameManager alone)

`CombatUI?.ShowCombatLog(...)` is called **605 times** in `GameManager.cs` and across **28 files** total. The formatting logic (emoji prefixes, color tags, string interpolation with stat lookups) is scattered inline.

Recommendation: Combat Log Helper

```
// NEW: CombatLogHelper.cs
public static class CombatLog
{
    public static void Attack(string attackerName, string targetName, int roll, int to
tal, int ac, bool hit)
        => CombatUI?.ShowCombatLog($"⚔ {attackerName} attacks {targetName}:
d20({roll}) + mods = {total} vs AC {ac} → {(hit ? "HIT" : "MISS")}");

    public static void SpellFailure(string casterName, string spellName, string reas-
on)
        => CombatUI?.ShowCombatLog($"⚡ {casterName}'s {spellName} fizzles! ({reas-
on})");

    public static void Damage(string targetName, int amount, string type)
        => CombatUI?.ShowCombatLog($"💥 {targetName} takes {amount} {type} damage.");

    public static void Warning(string message)
        => CombatUI?.ShowCombatLog($"⚠ {message}");
}
```

🟡 MEDIUM PRIORITY — Miss Chance / Concealment Logic

211 references to miss chance / concealment across the codebase. The logic for determining miss chance is spread across:

- `CharacterController.EvaluateEffectMissChanceAgainstAttacker()`
- `CharacterController.GetBlinkMissChanceAgainst()`
- `CharacterController.GetBlinkAttackerMissChance()`
- `GameManager.cs` (inline concealment checks during attacks)
- `CombatFlowService.cs`

Recommendation: Extract `ConcealmentResolver`

```
public static class ConcealmentResolver
{
    public struct ConcealmentResult
    {
        public int MissChance;
        public string Source; // "Blink", "Blur", "Displacement", "Darkness"...
        public bool DeniesTargetDex;
    }

    public static ConcealmentResult GetDefensiveConcealment(
        CharacterController defender, CharacterController attacker)
    { /* consolidate all miss-chance sources */ }

    public static ConcealmentResult GetOffensiveConcealment(
        CharacterController attacker)
    { /* attacker's own miss chance from Blink etc. */ }
}
```

---

3) Separation of Concerns Violations

🔴 GameManager.cs — The God Object (23,248 lines, 608 methods)

`GameManager.cs` handles **all** of the following (should be 10+ separate systems):

Responsibility	Approx. Lines	Should Be
Spell casting flow	~3,000	`SpellCastingService.cs`
Spell buff application	~2,000	`SpellEffectService.cs`
Attack/damage resolution	~2,000	Already partially in `CombatFlowService` — move more
Turn management	~500	Already in `TurnService` — complete extraction
Movement & pathfinding	~800	Already in `MovementService` — complete extraction
NPC AI orchestration	~1,500	Already in `AIService` — complete extraction
UI button handlers	~2,500	`ActionButtonService.cs` or directly in `ActionButtonPanel`
Summoning system	~600	`SummoningService.cs`
Concentration mechanics	~400	`ConcentrationManager.cs` (exists, but logic remains in GM)
Gold/loot management	~300	`EconomyService.cs`
Party setup/creation	~1,000	`PartySetupService.cs`
Character setup defaults	~800	Move to class files
Charge action	~300	`SupportActions.cs` (partially done)
Inline UI construction	118 UI calls	Move to `SceneBootstrap` / `UIFactory`

GameManager already uses partial classes (`GameManager_NewSpells.cs` , `GameManager_Grease.cs` , etc.), which shows awareness of the problem — but partial classes only split the file, they don't split responsibilities.

🟡 CharacterController.cs — Mixed Attack Logic + Status Effects (11,229 lines)

Contains:
- Attack calculations (single, full, dual-wield, natural, grapple) — ~3,000 lines
- Spell effect state (Blink, Ghoul Touch, Scare, False Life, etc.) — ~1,500 lines
- Movement logic — ~500 lines

- Grapple state machine — ~1,000 lines
- Feat property lookups — ~400 lines
- Equipment delegation — ~300 lines

Recommendation: Extract per-concern:

```
CharacterController.cs          → Core lifecycle, state, team, identity (~2,000 lines)
CharacterAttackResolver.cs      → All attack calculation methods (~3,000 lines)
CharacterSpellEffects.cs        → Blink, Ghoul Touch, Scare, etc. (~1,500 lines)
CharacterGrappleState.cs        → Grapple link, pin state, iterative grapple (~1,000
lines)
CharacterMovement.cs            → 5-foot step, movement validation (~500 lines)
```

These can remain as partial classes of `CharacterController` initially for zero-risk refactoring.

🟡 SceneBootstrap.cs — UI Construction + Wiring (1,470 lines)

Contains inline `new GameObject()` + `AddComponent<>()` UI construction. **187 UI construction calls**. This is a code-based UI builder (no Unity Editor scene files).

Recommendation:

Since this is a programmatic UI project, keep `SceneBootstrap` but extract per-panel factory methods:

```
UIFactory.cs                    → Already exists, expand it
SceneBootstrap_CombatUI.cs      → Partial class for combat UI construction
SceneBootstrap_Menus.cs         → Partial class for menu UI construction
```

---

4) Reusability Opportunities

🔴 Dice Rolling — No Centralized Service

`UnityEngine.Random.Range(1, 101)` and `Random.Range(1, 21)` are called **167 times** across the code-base with no abstraction. This makes:
- Testing impossible (can't mock rolls)
- Logging inconsistent
- Rule changes fragile

Recommendation: `DiceService`

```
public static class Dice
{
    public static int Roll(int sides) => UnityEngine.Random.Range(1, sides + 1);
    public static int D20() => Roll(20);
    public static int D100() => Roll(100);

    public static (int total, int[] rolls) RollMultiple(int count, int sides)
    {
        var rolls = new int[count];
        int total = 0;
        for (int i = 0; i < count; i++)
        {
            rolls[i] = Roll(sides);
            total += rolls[i];
        }
        return (total, rolls);
    }

    // For testing: inject a deterministic provider
    public static System.Func<int, int> RollProvider = (sides) => UnityEn-
gine.Random.Range(1, sides + 1);
}
```

🟡 Saving Throw Resolution — Scattered (191 references)

Saving throw logic (Will/Fort/Reflex + modifiers + conditions) is computed inline in many places rather than through a single resolver.

Recommendation: `SavingThrowResolver`

```
public static class SavingThrowResolver
{
    public struct SaveResult
    {
        public int Roll;
        public int Total;
        public int DC;
        public bool Success;
        public string SaveType; // "Will", "Fort", "Reflex"
    }

    public static SaveResult MakeSave(CharacterController target, string saveType,
int dc, string source)
    {
        int roll = Dice.D20();
        int bonus = saveType switch
        {
            "Will" => target.Stats.WillSave,
            "Fort" => target.Stats.FortSave,
            "Reflex" => target.Stats.ReflexSave,
            _ => 0
        };
        return new SaveResult { Roll = roll, Total = roll + bonus, DC = dc, Success =
(roll + bonus) >= dc, SaveType = saveType };
    }
}
```

🟡 Distance / Range Calculations

`SquareGridUtils` distance checks are called inline repeatedly. A `RangeHelper` that wraps common checks:

```
public static class RangeHelper
{
    public static bool IsWithinRange(CharacterController a, CharacterController b,
int rangeFeet)
        => SquareGridUtils.GetDistanceFeet(a.GridPosition, b.GridPosition) <= range-
Feet;

    public static bool IsAdjacent(CharacterController a, CharacterController b)
        => SquareGridUtils.GetDistanceSquares(a.GridPosition, b.GridPosition) == 1;

    public static bool IsWithinThreatRange(CharacterController attacker, CharacterCon-
troller target)
        => attacker.ThreatRange >= SquareG-
ridUtils.GetDistanceSquares(attacker.GridPosition, target.GridPosition);
}
```

---

5) Naming & Organization Suggestions

Current Folder Structure

```
Scripts/
├── AI/              (4 files + Profiles/ 13, Custom/ 1)
├── Character/       (47 files)          ← too large, mixed concerns
├── Classes/         (8 files)           ✅ good
├── Combat/          (28 files)          ← could split
├── CombatSystems/   (8 files)          ✅ good
├── Core/            (15 files)          ← GameManager bloat
├── Effects/         (5 files)           ✅ good
├── Grid/            (4 files)           ✅ good
├── Identifiers/     (11 files)          ✅ good
├── Inventory/       (8 files)           ✅ good
├── Magic/           (80 files!)         ← needs subfolder organization
├── Services/        (6 files)           ✅ good direction, needs more
├── Store/           (2 files)           ✅ good
├── Tests/           (multiple subdirs)  ✅ good
└── UI/              (29 files + subdirs)← needs more subfolders
```

Recommended Reorganization

```
Scripts/
├── AI/
│   ├── Profiles/
│   └── Custom/
├── Character/
│   ├── Stats/              ← CharacterStats.cs, CharacterConditions.cs
│   ├── Equipment/          ← CharacterEquipment.cs, CharacterTags.cs
│   ├── Feats/              ← FeatDefinitions.cs, FeatManager.cs
│   ├── Templates/          ← NPCDatabase.cs, NPCDatabaseCustom.cs, RaceDatabase.cs
│   └── Controller/         ← CharacterController + partial classes
├── Combat/
│   ├── Resolution/         ← CombatResult.cs, AttackCalculator.cs (NEW)
│   ├── Concealment/        ← ConcealmentResolver.cs (NEW)
│   ├── Maneuvers/          ← GrappleSystem, StandardManeuvers, OverrunSystem
│   └── Threat/             ← ThreatSystem.cs
├── Core/
│   └── (keep thin — just GameManager facade + bootstrap)
├── Magic/
│   ├── Spells/             ← SpellData.cs, SpellCaster.cs, SpellDatabase_*.cs
│   ├── AreaEffects/        ← All *AreaEffect.cs files
│   ├── StatusEffects/      ← All *EffectData.cs files
│   └── Systems/            ← AoESystem.cs, ConcentrationManager.cs
├── Services/               ← Expand: SpellCastingService, DiceService, etc.
└── UI/
    ├── Combat/             ← CombatUI, ActionButtonPanel, StatusEffectIndicator
    ├── Inventory/          ← PreCombatInventoryUI, InventoryUI, LootCollectionUI
    ├── CharacterCreation/  ← Already exists
    └── Panels/             ← Already exists
```

---

6) Priority-Ranked Refactoring Roadmap

Phase 1 — Quick Wins (Low Risk, High Impact) 🟢

#	Task	Impact	Risk	Effort
1	**Extract `CalculateFeatModifiers()`** from CharacterController — eliminate 3x duplication	High	Low	2-3 hrs
2	**Create `DiceService`** — wrap all `Random.Range` calls	Medium	Low	1-2 hrs
3	**Create `CombatLogHelper`** — standardize 605 log calls	Medium	Low	3-4 hrs
4	**Move Magic/area effects** into `Magic/AreaEffects/` subfolder	Low	None	30 min
5	**Move Magic/effect data** into `Magic/StatusEffects/` subfolder	Low	None	30 min

Phase 2 — Structural Splits (Medium Risk) 🟡

#	Task	Impact	Risk	Effort
6	**Split CharacterController** into partial classes by concern (attacks, spells, grapple, movement)	High	Medium	4-6 hrs
7	**Extract `SpellResolutionService`** from GameManager — unify PC/NPC spell casting	High	Medium	6-8 hrs
8	**Extract `SpellBuffService`** from GameManager — all `Apply*Buff()` methods	High	Medium	4-6 hrs
9	**Move button handlers** from GameManager to ActionButtonPanel or new service	Medium	Medium	4-6 hrs
10	**Extract `SavingThrowResolver`**	Medium	Low	2-3 hrs
11	**Extract `ConcealmentResolver`**	Medium	Low	3-4 hrs

Phase 3 — Architecture (Higher Risk, Long-term) 🔴

#	Task	Impact	Risk	Effort
12	**Slim GameManager to facade** — delegate to services, keep only orchestration	Very High	High	2-3 days
13	**Create `SummoningService`** — extract ~600 lines from GameManager	Medium	Medium	4-6 hrs
14	**Create `PartySetupService`** — extract character creation/setup	Medium	Medium	4-6 hrs
15	**Introduce event bus** for cross-system communication (replace direct CombatUI calls)	High	High	2-3 days

---

7) Summary Statistics

Metric	Value
Total C# files	355
Total lines	164,603
Files over 1,000 lines	27
Files over 2,000 lines	10
Largest file	GameManager.cs (23,248 lines)
GameManager total (partials)	28,437 lines
Feat-bonus duplication	3x (33 checks duplicated)
Combat log calls (GameManager)	605
Miss chance references	211
Saving throw references	191
Random.Range calls	167
Files using ShowCombatLog	28

Key Takeaway

The project has **solid domain modeling** (SpellNames constants, FeatManager, ConditionService, StatusEffectManager) and **good initial service extraction** (CombatFlowService, TurnService, MovementService, AIService). The main debt is concentrated in two files — **GameManager.cs** and **CharacterController.cs** — which together account for **34,477 lines (21% of the entire codebase)**. The partial class pattern is already used, making incremental extraction safe and low-risk. The highest-ROI refactoring is **extracting the feat modifier calculation** (eliminates 3x duplication in the most critical code path) followed by **unifying PC/NPC spell resolution** (eliminates the most dangerous duplication — divergent copies of spell failure logic).